

We now want to give a formal account of the type-theoretic presentation of SSA we sketched in the introduction. In particular, for each of the intermediate representations we wish to study, we need to provide:

- A *grammar*, to enumerate the syntactically well-formed terms in our language – we’ve already provided this in the introduction for most of our intermediate representations.
- A *typing relation*, which allows us to determine whether a program is *semantically* well-formed.

In particular, we’ll need a *type system* to construct our typing relation with respect to – later, we’ll be able to relate different intermediate representations by giving them different typing relations over the same type system.

S-Expressions

Language

We'll start by giving a formal account of the simplest practical language – that of *first-order* expressions like $x + y$ and $a + (5 * b) + \sin(3)$.

By *first-order*, we mean that expressions can't return functions for further application – for example, we can't write $\text{add}(1)(2)$ to mean “apply the function $\lambda x.(x + 1)$ to 2” – instead, we've got to stick with the simple function applications and arithmetic expressions most familiar to grade-schoolers and imperative programmers.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n -ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f e_1 \dots e_n)$.

We can then uniformly write binary operations like $x + y$ as binary sexprs $(+ x y)$, unary operations $-x$ as unary sexprs $(- x)$, and function applications $f(x, y, z)$ as n -ary sexprs $(f x y z)$. In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit.

Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, f can't be a compound expression itself (like in $((\text{add } 1) 2)$).

More formally:

Fixing

- A set of *variables* $x \in N$
- A set of *operations* $f \in O$
- A set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \text{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f e^*)$$

Given a set of *types* $A \in \mathcal{T}$, we'll define a *stack-typing relation* on O to be any relation

$$f : [A_1, \dots, A_m] \rightarrow [B_1, \dots, B_n]$$

where

- $A_1, \dots, A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f .
- $B_1, \dots, B_n \in \mathcal{T}$ are types – n is called the *output arity* of f .

For now, we'll only consider operations with an output arity of 1.

We define a (*typing*) *context* $\Gamma \in \text{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, \dots, x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says “under the assumptions of Γ , the S-expression e has type A ”. The rules are:

$$\text{var-head} \frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail} \frac{\Gamma \vdash y : B, x \neq y}{\Gamma, x : A \vdash y : B}$$

$$\text{app} \frac{f : [A_1, \dots, A_n] \rightarrow [B] \quad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f \ a_1 \ \dots \ a_n) : B}$$

Sequencing

Branching

Loops

Unstructured Control-Flow

